COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE212 / Automating Server Management**	**SECTION:**	**CPE31S2**
DATE PERFORMED:	**October 12 2026**	**DATE SUBMITTED:**	**October 12 2026**
NAME:	**Kenzo Lim**	**INSTRUCTOR:**	**Engr. Jimlord Quejado**

ACTIVITY NO. 11.1 Containerization

1. PROCEDURE OUTPUT

1. Create a new repository for this activity.
2. Install Docker and enable the docker socket.
3. Add to Docker group to your current user.
4. Create a Dockerfile to install web and DB server.
5. Install and build the Dockerfile using Ansible.
6. Add, commit and push it to your repository.

Created a directory for HOA 11

```
kenzo@workstation:~/ansible_proj/CPE212_Lim$ mkdir HOA11
kenzo@workstation:~/ansible_proj/CPE212_Lim$ cd HOA11
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$
```

I created the HOA11 folder inside my CPE212_Lim repository using mkdir and moved into it with cd

Installing Docker.io

```
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ sudo apt install -y docker.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bridge-utils containerd pigz runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools btrfs-progs cgroupfs-mount | cgroup-lite debootstrap
  docker-buildx docker-compose-v2 docker-doc rinse zfs-fuse | zfsutils
The following NEW packages will be installed:
  bridge-utils containerd docker.io pigz runc ubuntu-fan
0 upgraded, 6 newly installed, 0 to remove and 25 not upgraded.
Need to get 73.7 MB of archives.
After this operation, 278 MB of additional disk space will be used.
Get:1 http://ph.archive.ubuntu.com/ubuntu jammy/universe amd64 pigz amd64 2.6-1 [63.6 kB]
Get:2 http://ph.archive.ubuntu.com/ubuntu jammy/main amd64 bridge-utils amd64 1.7-1ubuntu3 [34.4 kB]
Get:3 http://ph.archive.ubuntu.com/ubuntu jammy-updates/main amd64 runc amd64 1.3.4-0ubuntu1~22.04.1 [
,569 kB]
14% [3 runc 6,789 kB/9,569 kB 71%]                                    1,092 kB/s 1min14
Get:4 http://ph.archive.ubuntu.com/ubuntu jammy-updates/main amd64 containerd amd64 2.2.1-0ubuntu1~22.
4.2 [28.3 MB]
20% [4 containerd 0 B/28.3 MB 0%]                                     1,092 kB/s 58s
```

I installed Docker using sudo apt install -y docker.io. The output shows apt downloading and installing Docker along with its dependencies such as containerd and runc.

Enabling docker socket and service

```
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ sudo systemctl enable --now docker.socket
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ sudo systemctl enable --now docker.service
```

I ran systemctl enable --now on docker.socket and docker.service so both start immediately and automatically on every boot.

Verifying docker.socket status

```
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ sudo systemctl status docker.socket
● docker.socket - Docker Socket for the API
     Loaded: loaded (/lib/systemd/system/docker.socket; enabled; vendor preset: enabled)
     Active: active (running) since Mon 2026-10-12 15:02:06 PST; 1min 52s ago
   Triggers: ● docker.service
     Listen: /run/docker.sock (Stream)
      Tasks: 0 (limit: 4598)
     Memory: 0B
        CPU: 791us
     CGroup: /system.slice/docker.socket

Oct 12 15:02:06 workstation systemd[1]: Starting Docker Socket for the API...
Oct 12 15:02:06 workstation systemd[1]: Listening on Docker Socket for the API.
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$
```

The status output shows docker.socket as active (running) and listening on /run/docker.sock, which confirms the Docker API socket is enabled.

Adding the current user to the docker group and verifying

```
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ sudo usermod -aG docker $USER
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ newgrp docker
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ groups
docker adm cdrom sudo dip plugdev lpadmin lxd sambashare kenzo
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/
```

I added my user to the docker group with usermod -aG and applied it using newgrp docker. The groups output now lists docker, and docker run hello-world ran successfully without sudo, which proves Docker works for my user.

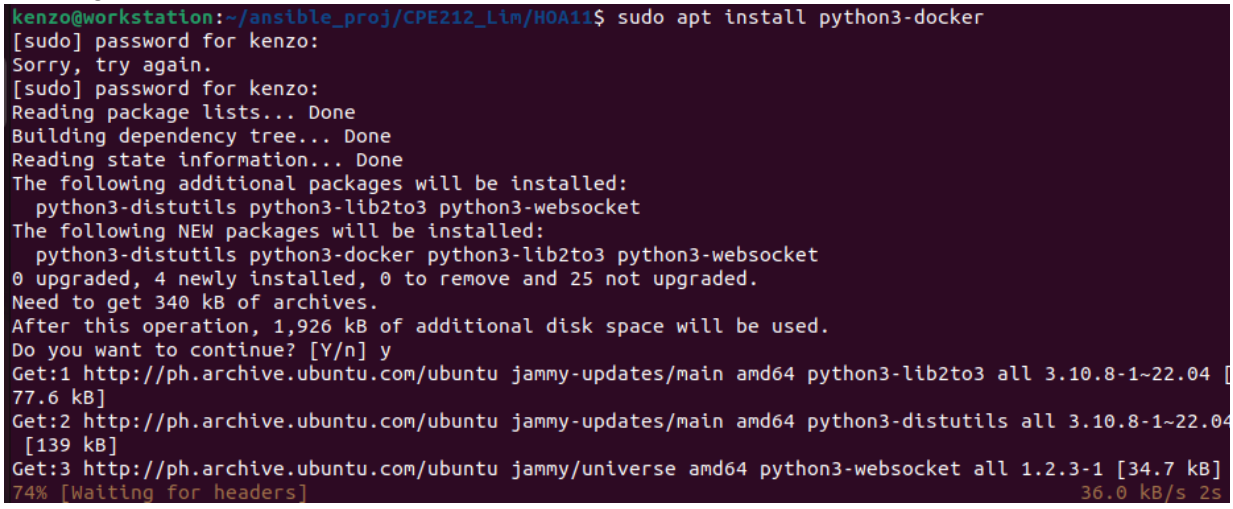
Creating the dockerfile (Web + DB Server)





The first screenshot is start.sh, the startup script that launches MariaDB and then Apache in the foreground. The second is the Dockerfile, which uses Ubuntu 22.04, installs apache2 and mariadb-server, copies in start.sh, exposes ports 80 and 3306, and runs the script on startup.

Installing docker modules



```
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ ansible-galaxy collection install community.docker
Starting galaxy collection install process
Nothing to do. All requested collections are already installed. If you want to reinstall them, consider
 using `--force`.
```

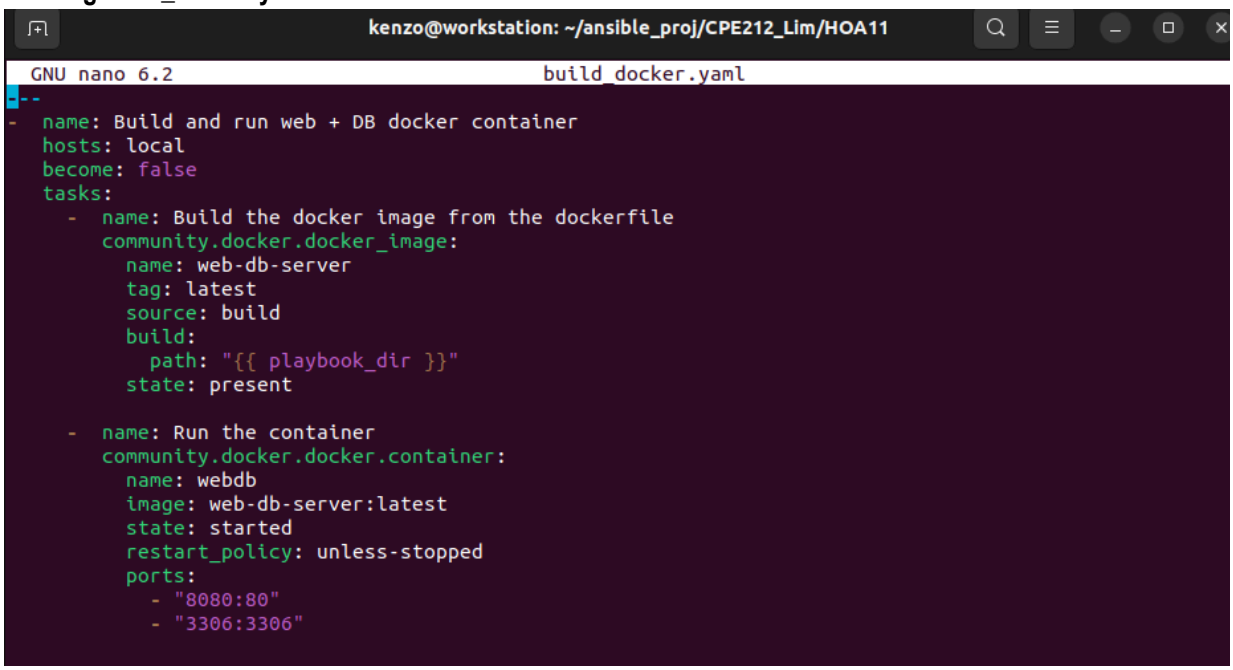
I installed python3-docker with apt so Ansible can communicate with Docker. The ansible-galaxy output shows the community.docker collection was already installed.

Creating inventory file



The inventory.ini file defines a [local] group containing localhost with a local connection, so the playbook runs on my own workstation.

Creating build_docker.yaml



This playbook has two tasks: build the web-db-server image from the Dockerfile in the playbook directory, then run it as a container named webdb with ports 8080:80 and 3306:3306 and a restart policy.

Running the playbook build_docker.yaml

```
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ ansible-playbook -i inventory.ini build_docker.yaml

PLAY [Build and run web + DB docker container] *********************************************

TASK [Gathering Facts] *********************************************************************
[WARNING]: Platform linux on host localhost is using the discovered Python interpreter at
/usr/bin/python3.10, but future installation of another Python interpreter could change the meaning of
that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]

TASK [Build the docker image from the dockerfile] ******************************************
changed: [localhost]

TASK [Run the container] *******************************************************************
changed: [localhost]

PLAY RECAP *********************************************************************************
localhost                  : ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
```

I ran ansible-playbook -i inventory.ini build_docker.yaml. The recap shows ok=3 changed=2 failed=0, meaning the image was built and the container was started without errors.

Verifying the playbook worked

```
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ docker images
                                                                    i Info →  U  In Use
IMAGE                  ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest     5e2309035332       25.9kB         9.49kB    U
ubuntu:22.04           5ec03bb3441e        119MB         31.7MB
web-db-server:latest   168cd0693887        616MB          116MB    U
```

```
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ docker ps
CONTAINER ID   IMAGE                  COMMAND       CREATED          STATUS          PORTS
                                      NAMES
55190825fef1   web-db-server:latest   "/start.sh"   18 seconds ago   Up 17 seconds   0.0.0.0:
3306->3306/tcp, 0.0.0.0:8080->80/tcp   webdb
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ curl http://localhost:8080
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/D
TD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
  * {
    margin: 0px 0px 0px 0px;
    padding: 0px 0px 0px 0px;
  }

  body, html {
    padding: 3px 3px 3px 3px;

    background-color: #D8DBE2;
```

```
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ docker exec webdb mysql -e "SELECT VERSION
();"
VERSION()
10.6.23-MariaDB-0ubuntu0.22.04.1
```

docker images lists the new web-db-server:latest image, and docker ps shows the webdb container as Up with ports 8080 and 3306 mapped. curl http://localhost:8080 returns the Apache2 Ubuntu Default Page, and SELECT VERSION(); returns MariaDB 10.6.23, so both the web and DB servers are working.

Dockerfile and start.sh contents modified due to errors (restarting status of the container)

```
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ cat Dockerfile
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && \
    apt-get install -y apache2 mariadb-server && \
    apt-get clean && rm -rf /var/lib/apt/lists/*

COPY start.sh /start.sh
RUN chmod +x /start.sh

EXPOSE 80 3306

CMD ["/start.sh"]
kenzo@workstation:~/ansible_proj/CPE212_Lim/HOA11$ cat start.sh
#!/bin/bash
source /etc/apache2/envvars
mkdir -p "$APACHE_RUN_DIR" "$APACHE_LOCK_DIR" "$APACHE_LOG_DIR"
chown www-data:www-data "$APACHE_RUN_DIR" "$APACHE_LOCK_DIR" "$APACHE_LOG_DIR"
service mariadb start
exec /usr/sbin/apache2 -D FOREGROUND
```

The container was stuck in a restarting state because the original start.sh used apachectl, which does not exist in the image. I changed it to source /etc/apache2/envvars, create and set ownership of the Apache run, lock, and log directories, start MariaDB, then run /usr/sbin/apache2 -D FOREGROUND.

Commit and push after adding

```
kenzo@workstation:~/ansible_proj/CPE212_Lim$ git add HOA11
kenzo@workstation:~/ansible_proj/CPE212_Lim$ git commit -m "HOA11 Containerization"
[main 12bf267] HOA11 Containerization
 4 files changed, 45 insertions(+)
 create mode 100644 HOA11/Dockerfile
 create mode 100644 HOA11/build_docker.yaml
 create mode 100644 HOA11/inventory.ini
 create mode 100755 HOA11/start.sh
kenzo@workstation:~/ansible_proj/CPE212_Lim$ git push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 2 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (7/7), 1.09 KiB | 139.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:qknlim/CPE212_Lim
   36c17c4..12bf267  main -> main
```

In this last part, I added HOA11 and committed it by the message “HOA11 Containerization” and pushed it into the main branch.

Github Link: https://github.com/qknlim/CPE212_Lim

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Reflections:
Answer the following:
1. What are the benefits of implementing containerizations?
Containerization packages an application with its dependencies so it runs consistently on any machine. Containers start quickly and use fewer resources than virtual machines. They isolate applications to reduce conflicts and make deployment easier.

Conclusion:
In this activity, I was able to install Docker, enable its socket, and add my user to the docker group so I could manage containers without root privileges. I wrote a Dockerfile that installs Apache and MariaDB, then used an Ansible playbook to build the image and run the container. The container first kept restarting because my start.sh called apachectl, which does not exist in the image, and because Apache’s run directory was missing. After I changed start.sh to load Apache’s environment variables, create the required directories, and run /usr/sbin/apache2 in the foreground, the container stayed up. Both the Apache default page and the MariaDB version query worked. Overall, I realized that reading docker logs was my important reflection in this activity as it helped in fixing the errors I encountered.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Error analysis and code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Claude to troubleshoot why the container kept restarting and to get suggestions for the Dockerfile, start.sh, and Ansible playbook.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.’s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.